Unidad 6
APIs, HTTP, APIs Web & JSON

Materia: Programación 3 Frontend
Año: 2026
Profesor: Lucas Ferrero

UTN FACULTAD REGIONAL RAFAELA

¿Qué es una API?

Una API (Application Programming Interface) es una interfaz que permite la comunicación entre distintos componentes de software, ya sea en una misma máquina o de forma distribuida.

Ejemplos comunes de APIs incluyen:

- APIs de librerías → Funciones que utilizan dentro de un mismo programa (ej: Math en Java, stdlib.h en C)
- APIs de sistemas operativos → Permiten a los programas interactuar con el hardware y el SO
- **APIs web** → Permiten la comunicación entre aplicaciones a través de internet usando el protocolo HTTP

Protocolos de Comunicación

Un protocolo, en informática, es un conjunto de reglas que permiten el intercambio de datos entre computadoras. Especifica cómo se debe estructurar y transmitir la información.

Ejemplos de protocolos:

- **HTTP / HTTPS** (Hypertext Transfer Protocol) → Transferencia de datos en la web
- TCP/IP → Base de toda comunicación en internet
- FTP → Transferencia de archivos
- SMTP → Envío de correos electrónicos

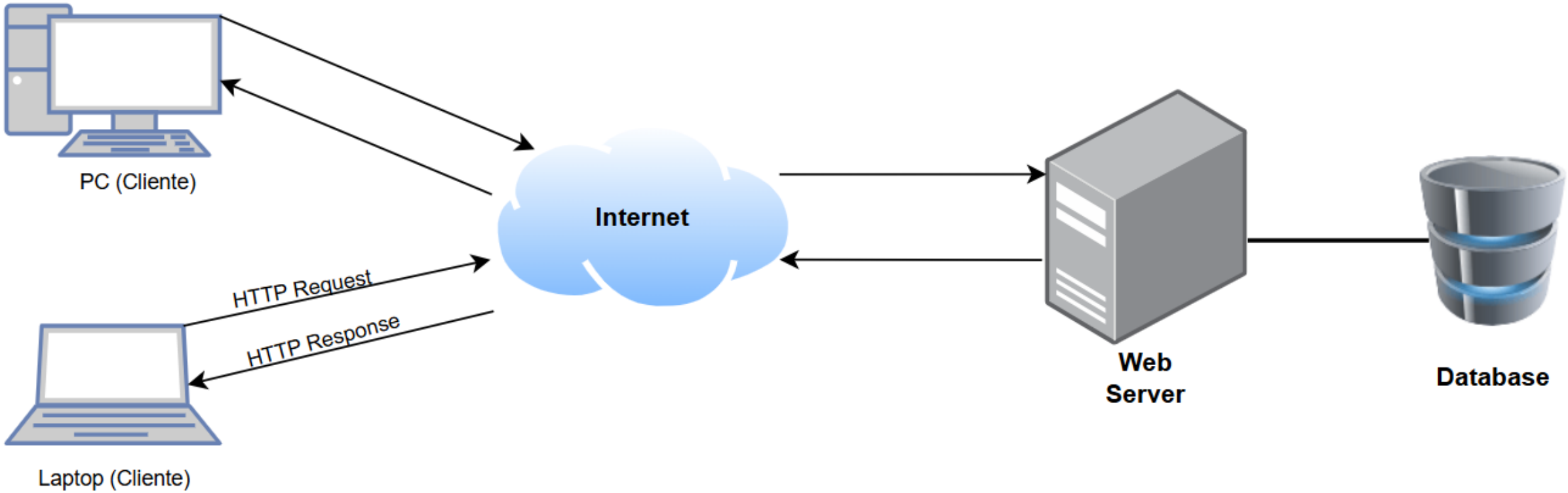
Un protocolo es un conjunto de reglas que define cómo deben comunicarse dos partes para que se entiendan. Sin un protocolo en común, la comunicación es imposible.

HTTP: HyperText Transfer Protocol

HTTP es el protocolo que define cómo se comunican clientes y servidores en la web. Funciona bajo un modelo **Request → Response**: el cliente siempre inicia la comunicación y el servidor siempre responde.

Características principales:

- **Stateless (Sin estado)**: Cada request es independiente. El servidor no recuerda requests anteriores. Si se envían dos requests seguidas, el servidor los trata como si fueran distintas.
- **Basado en texto**: Los mensajes HTTP son texto plano, lo que los hace legibles y fáciles de debuggear.
- **Métodos (verbos)**: Indican la intención del request (GET, POST, PUT, PATCH, DELETE)
- **Códigos de estado**: Indican el resultado del request (200, 404, 500, etc.)
- **Headers**: Metadatos que acompañan al request o response
- **Body**: Contenido del mensaje (no esta siempre presente)


PC (Cliente)
Internet
HTTP Request
HTTP Response
Laptop (Cliente)
Web
Server
Database

Códigos de Estado HTTP

Los códigos de estado indican el resultado de una solicitud. Se agrupan en familias según el primer dígito:

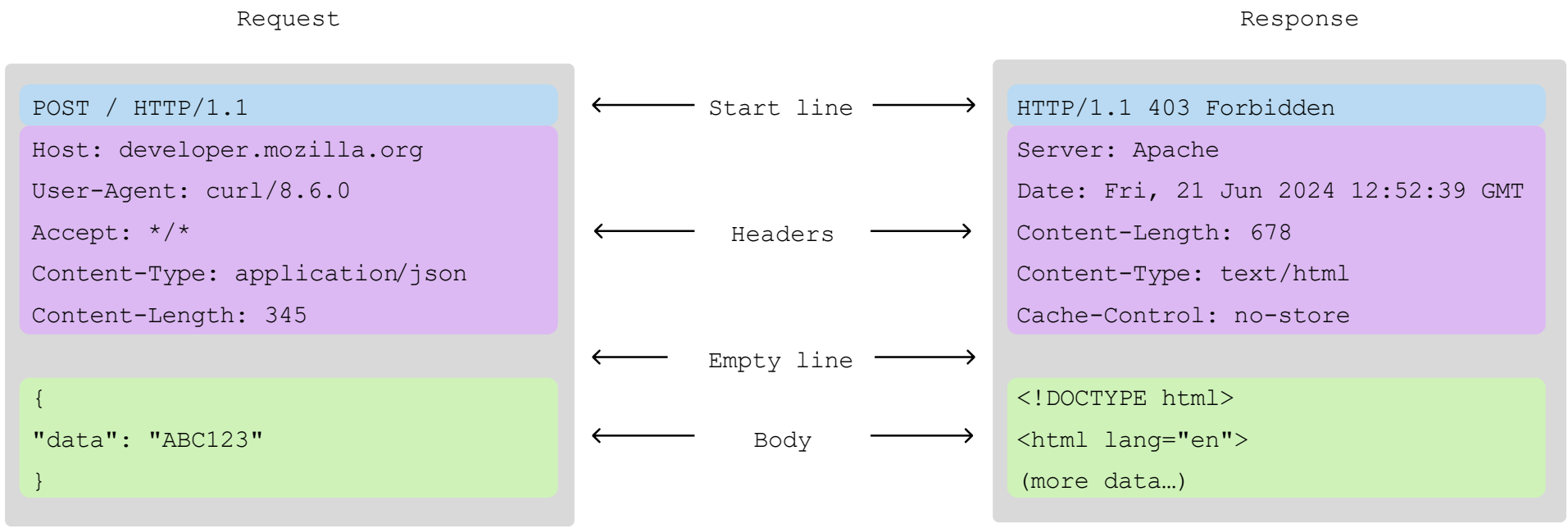
Familia	Significado	Códigos importantes
1xx	Informacional	Raramente se ve en APIs
2xx	Éxito	200 OK, 201 Created, 204 No Content
3xx	Redirección	301 Moved Permanently
4xx	Error del cliente	400, 401, 403, 404, 422
5xx	Error del servidor	500 Internal Server Error, 503

Códigos más comunes en detalle:

Código	Nombre	Cuándo ocurre
200	OK	Request exitoso, hay datos en el response
201	Created	Se creó un recurso exitosamente (POST)
204	No Content	Éxito pero sin datos en el response (DELETE)
400	Bad Request	El cliente envió datos mal formados
401	Unauthorized	No se enviaron las credenciales (o son inválidas)
403	Forbidden	Estás autenticado pero no tenés permiso
404	Not Found	El recurso no existe
500	Internal Server Error	Algo falló en el servidor

Anatomía de un Request y un Response HTTP

Todo mensaje HTTP tiene una estructura fija. Conocerla nos permite entender exactamente qué pasa en cada comunicación.



REQUEST (lo que envía el cliente):

```
POST /productos HTTP/1.1
Host: api.mitienda.com
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

{
  "nombre": "Laptop",
  "precio": 1500
}
```

Línea 1: Método + URL + Versión HTTP

Líneas 2-4: Headers (metadatos)

Línea en blanco: Separador obligatorio entre headers y body

Bloque final: Body con los datos

RESPONSE (lo que devuelve el servidor):

```
HTTP/1.1 201 Created
Content-Type: application/json

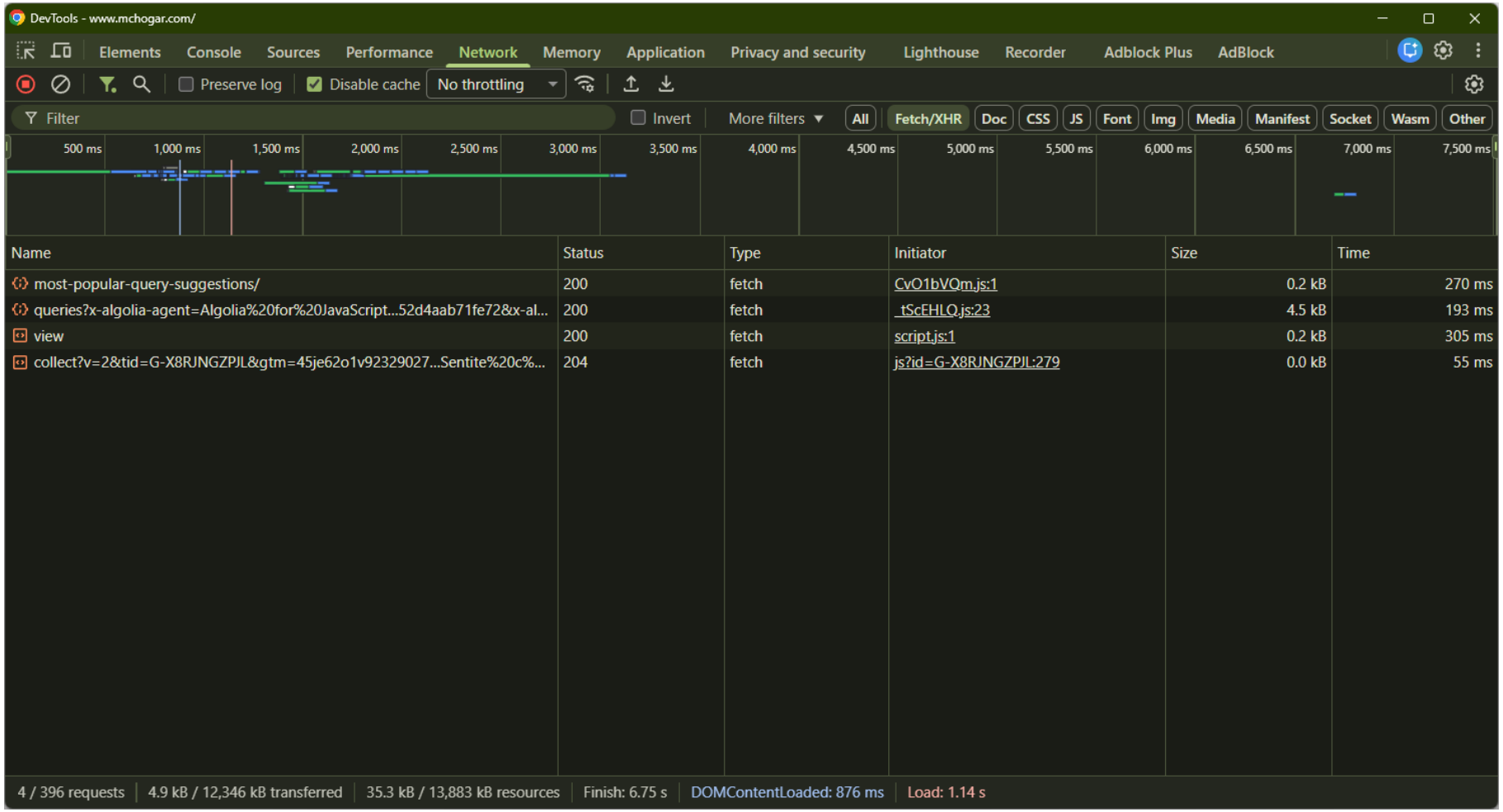
{
  "id": 42,
  "nombre": "Laptop",
  "precio": 1500
}
```

Línea 1: Versión HTTP + Código de estado + Mensaje

Línea 2: Headers de la respuesta

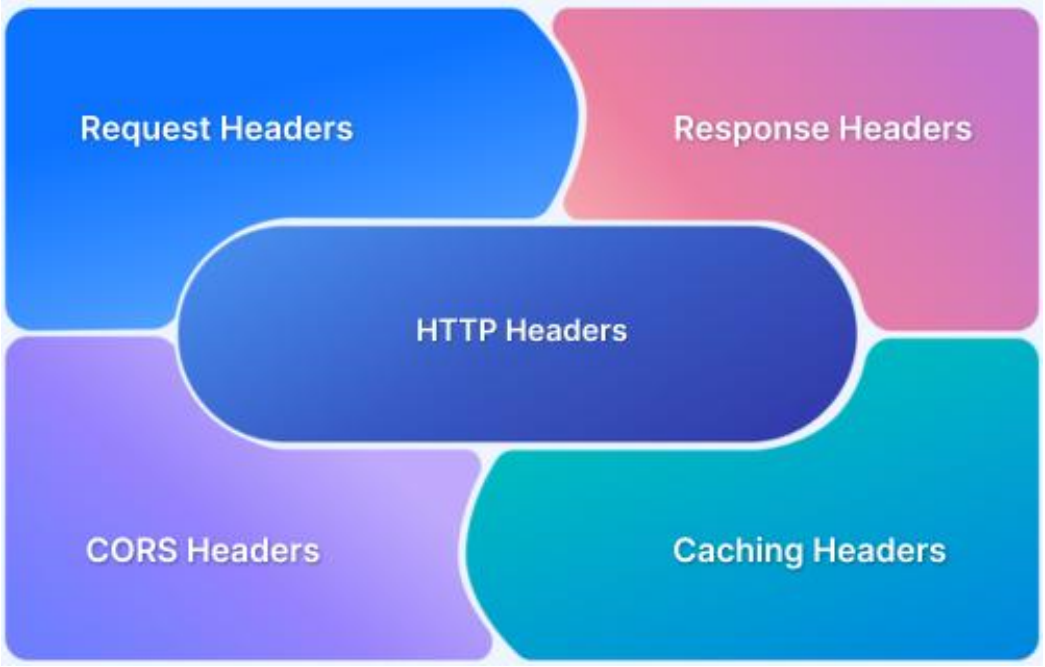
Bloque final: Body con los datos devueltos

Cuando algo falla en una integración con una API, es recomendado inspeccionar el request y el response crudos. En el navegador podés verlos en DevTools → pestaña Network.



Headers HTTP — Los metadatos del mensaje

Los **headers (cabeceras)** son pares clave-valor que acompañan a requests y responses. No son el contenido del mensaje, sino información sobre el mensaje.



Headers en el Request:

Header	Ejemplo	Para qué sirve
Content-Type	application/json	Le dice al servidor el formato del body que estamos enviando
Authorization	Bearer <token>	La credencial de identidad en cada request
Accept	application/json	Pedir al servidor en qué formato queremos la respuesta

Headers en el Response:

Header	Ejemplo	Para qué sirve
Content-Type	application/json	Nos indica el formato del body que recibimos
Status	200 OK	Resultado de la operación

API Web / Web Service y REST

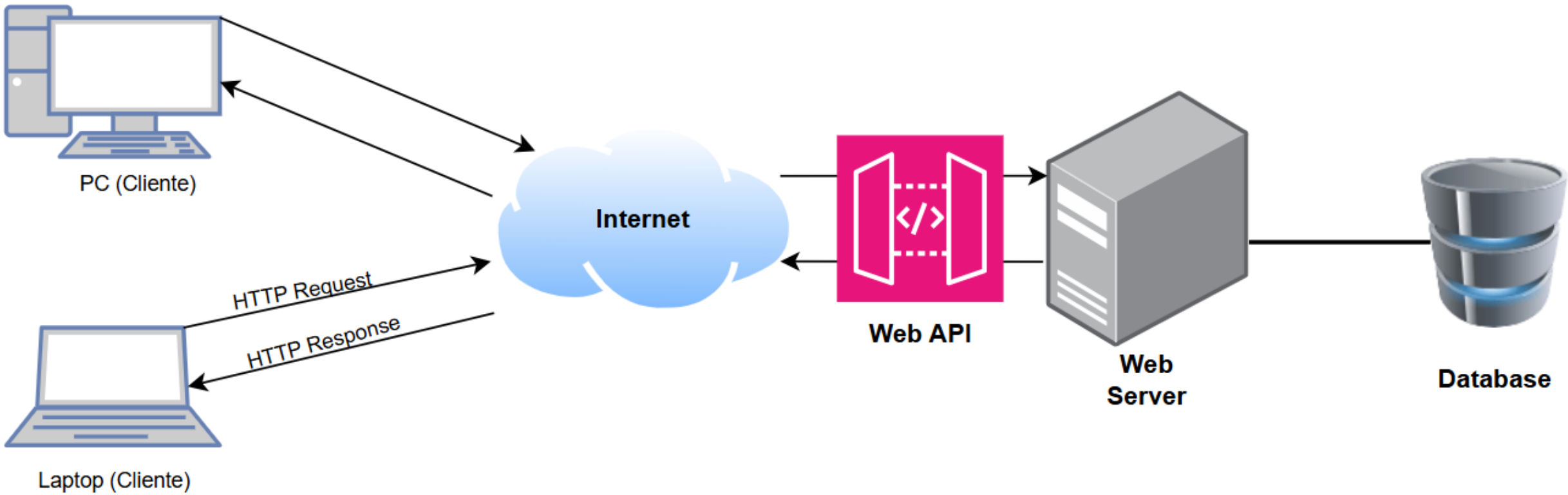
Una API Web es una interfaz que permite que aplicaciones cliente interactúen con servidores a través de internet usando HTTP.

Características de una API Web:

- Basada en recursos identificados por URLs
- Usa **métodos HTTP** para operar sobre esos recursos
- **Stateless**: cada solicitud es independiente.
- Las respuestas usan formatos estructurados, generalmente JSON

Ejemplo de endpoints:

- GET /productos → devuelve la lista de productos.
- GET /productos/5 → devuelve el producto con ID 5
- POST /usuarios -> crea un nuevo usuario.
- POST /usuarios → crea un nuevo usuario
- DELETE /productos/5 → elimina el producto con ID 5

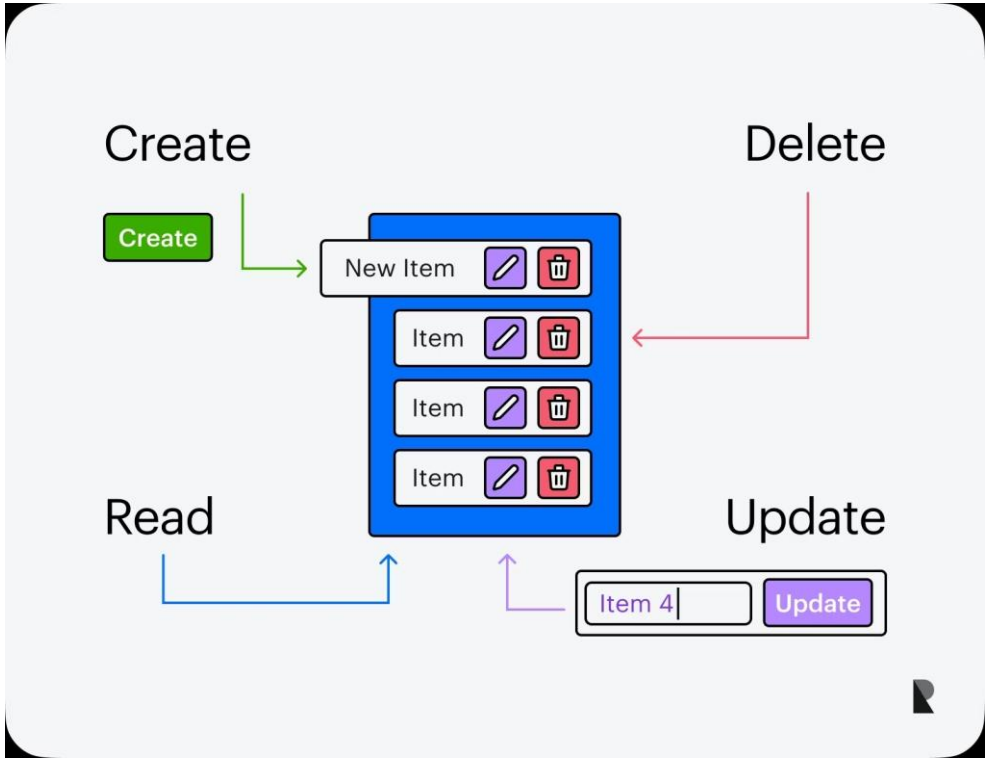

PC (Cliente)
Internet
HTTP Request
HTTP Response
Laptop (Cliente)
Web API
Web
Server
Database

Ejemplo de interacción con una API Web

1. El cliente realiza una solicitud HTTP (ej. GET /productos)
2. La API Web recibe la solicitud y la interpreta.
3. Servidor procesa la solicitud (lógica de negocio y acceso a base de datos).
4. El servidor construye y envía una respuesta en formato JSON.
5. El cliente recibe y procesa la respuesta.

CRUD (ABM/ABML)

CRUD es un acrónimo de “Create”, “Read”, “Update”, “Delete”, que se usa para referirse a las funciones básicas que posee un sistema.



Métodos HTTP más utilizados

También llamados verbos HTTP, indican la intención de la solicitud al servidor.

Método	Propósito	CRUD Relacionado	Descripción
GET	Obtener información	Read	Solicita datos desde el servidor. No modifica nada.
POST	Crear un recurso	Create	Envía datos al servidor para crear algo nuevo.
PUT	Reemplazar completamente un recurso	Update	Actualiza un recurso existente (reemplazo total).
PATCH	Modificar parcialmente un recurso	Update	Cambia parcialmente un recurso (campo específico).
DELETE	Eliminar un recurso	Delete	Borra un recurso del servidor.

Relación con CRUD

Operación CRUD	Método HTTP
Create	POST
Read	GET
Update	PUT / PATCH
Delete	DELETE

El Body en las peticiones HTTP

El body (cuerpo) es la parte de un request o response donde se envía el contenido principal del mensaje.

¿Cuándo se usa el body?

Método	¿Usa Body en el Request?	¿Por qué?
GET	No	Solo pide datos, no envía nada
POST	Sí	Envía los datos del recurso a crear
PUT	Sí	Envía el recurso completo a reemplazar
PATCH	Sí	Envía solo los campos a modificar
DELETE	No (usualmente)	El ID va en la URL, no en el body

Formato del body: El contenido del body puede tener diferentes formatos. El más común hoy en día es JSON. El servidor sabe qué formato recibe gracias al header *Content-Type*.

Ejemplo de body en un POST:

```json
{
  "nombre": "Laptop",
  "precio": 1500,
  "categoria": "tecnologia"
}
```

JSON

JSON significa JavaScript Object Notation. Es un formato de texto ligero para el intercambio de datos. Se utiliza para estructurar información de manera que sea fácilmente entendible por humanos y máquinas.

Características principales

- Sintaxis basada en JavaScript, pero es independiente del lenguaje (lo entienden casi todos los lenguajes modernos: Python, Java, C#, PHP, etc.).
- Representa objetos y arreglos usando texto plano.
- Es el formato más común de intercambio de datos en APIs Web (por ejemplo, al consumir o enviar datos desde un frontend a un backend vía HTTP).

Estructura de JSON

JSON puede contener:

Tipos de valores: string, number, boolean, null, objeto (con {}), arreglo (con []).



```json
{
  "nombre": "Notebook HP",
  "precio": 500000,
  "en_stock": true,
  "caracteristicas": {
    "procesador": "Intel i5",
    "memoria": "16GB"
  },
  "etiquetas": ["oferta", "nueva", "envio gratis"]
}
```

¿Dónde se usa JSON?

- En cuerpos de solicitudes HTTP (como POST o PUT).
- En respuestas de APIs REST.
- En configuraciones (package.json, tsconfig.json, firebase.json, etc.).
- En almacenamiento NoSQL (como MongoDB, DynamoDB o Firestore)

JSON y objetos Javascript

Aunque la sintaxis es similar, un JSON es un string con un determinado formato, mientras que un objeto en JavaScript es una estructura viva en memoria.

Un objeto JavaScript no es lo mismo que un JSON.

Ejemplos peticiones a una API Web

Supongamos una API de productos.

Estos serían ejemplos típicos de requests:

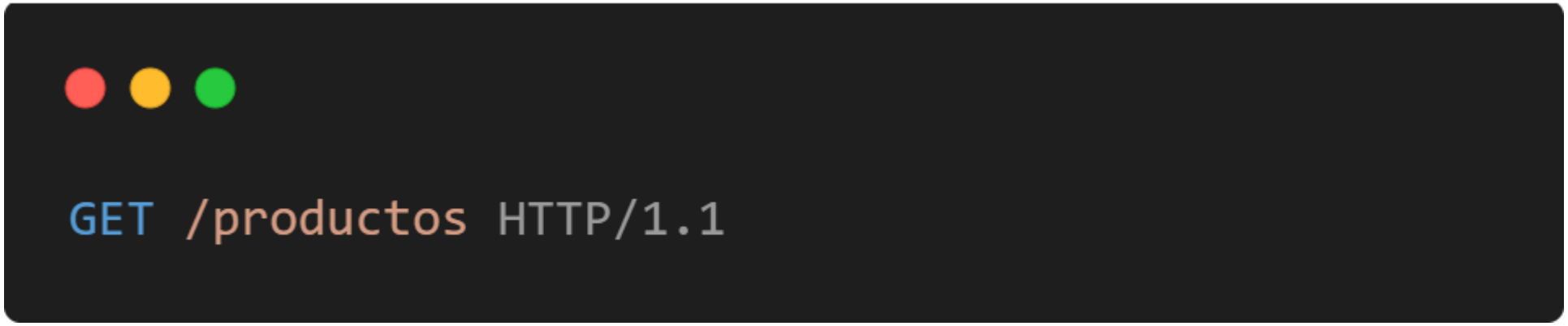
POST /productos

```
POST /productos HTTP/1.1
Content-Type: application/json

{
  "nombre": "Notebook HP",
  "precio": 500000,
  "stock": 12
}
```

Crea un nuevo producto.

GET /productos



Devuelve la lista de productos (no le enviamos nada en body).

PUT /productos/123

```
PUT /productos/123 HTTP/1.1
Content-Type: application/json

{
  "nombre": "Notebook HP",
  "precio": 480000,
  "stock": 10
}
```

Reemplaza completamente el producto con ID 123.

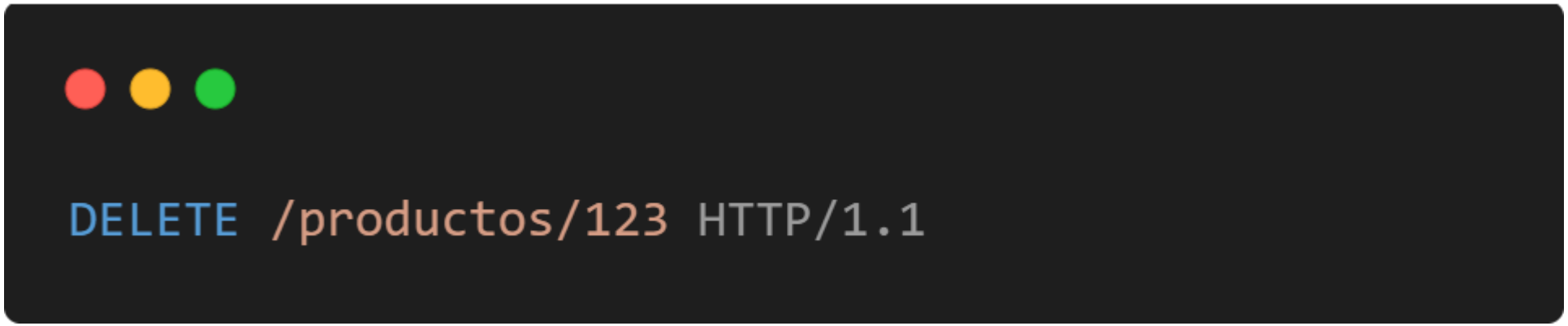
PATCH /productos/123

```
PATCH /productos/123 HTTP/1.1
Content-Type: application/json

{
  "precio": 460000
}
```

Modifica únicamente el precio del producto 123.

DELETE /productos/123



Elimina el producto con ID 123.

Eliminación Lógica vs Eliminación Física

Cuando trabajamos con APIs y bases de datos, eliminar un recurso no siempre significa borrarlo físicamente.

Existen dos enfoques:

Hard Delete (Eliminación Física)

- El recurso se elimina físicamente de la base de datos.
- No se puede recuperar (o no es fácil hacerlo).

Ejemplo

Http:

```
DELETE /productos/123 HTTP/1.1
```

Internamente en SQL:

```
DELETE FROM productos WHERE id = 123;
```

Ventaja: Ahorra espacio y mantiene la db limpia.

Desventaja: Puede perderse información importante (ej. auditoría o historial).

Soft Delete (Eliminación Lógica)

El recurso no se borra, sino que se marca como eliminado usando un campo adicional (ej. *eliminado = true* o *deleted_at* con una fecha).

El dato sigue en la base pero no se muestra más al usuario final.

Ejemplo:

```sql
UPDATE productos SET eliminado = true WHERE id = 123;
```

La API puede seguir usando DELETE /productos/123, pero internamente se hace un UPDATE.

Ventaja: Permite recuperar información o realizar auditorías.

Desventaja: Requiere lógica adicional para "ignorar" registros eliminados lógicamente en consultas normales.

Serialización y Deserialización

¿Qué es Serialización?

La serialización es el proceso de convertir un objeto en memoria (de un lenguaje de programación) en un formato que se pueda almacenar o transmitir, como por ejemplo un JSON.

En otras palabras: transformar estructuras complejas (objetos) en datos planos para poder enviarlos por la red o guardarlos.

¿Qué es Deserialización?

La deserialización es el proceso inverso: tomar un JSON y convertirlo de nuevo en un objeto o estructura de datos que pueda usarse dentro de un programa.

Ejemplo con JSON

JSON:

```json
{
  "nombre": "Notebook HP",
  "precio": 500000,
  "en_stock": true
}
```

Javascript

```javascript
const producto = {
  nombre: "Notebook HP",
  precio: 500000,
  en_stock: true
};

// Serialización
const jsonString = JSON.stringify(producto);
console.log(jsonString); // Resultado: '{"nombre":"Notebook HP","precio":500000,"en_stock":true}'


// Deserialización
const parsedObject = JSON.parse(jsonString);
console.log(parsedObject.nombre); // "Notebook HP"
```

C# (.NET)

```
using System.Text.Json;

// Clase
public class Producto
{
    public string Nombre { get; set; }
    public int Precio { get; set; }
    public bool EnStock { get; set; }
}

// Serializar
Producto producto = new Producto { Nombre = "Notebook HP", Precio = 500000, EnStock = true };
string json = JsonSerializer.Serialize(producto);
Console.WriteLine(json); // Resultado: {"Nombre":"Notebook HP","Precio":500000,"EnStock":true}

// Deserializar
Producto nuevoProducto = JsonSerializer.Deserialize<Producto>(json);
Console.WriteLine(nuevoProducto.Nombre); // Notebook HP
```

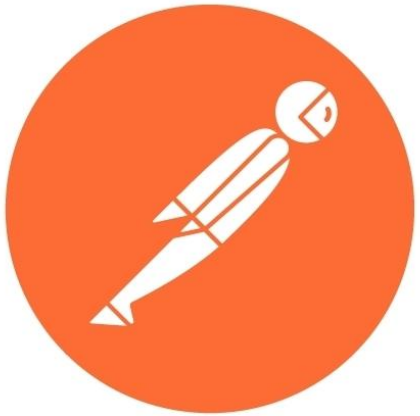
Postman — Herramienta para trabajar con APIs

Postman es una herramienta estándar de la industria para probar, documentar e interactuar con APIs Web, sin necesidad de escribir código. Actúa como un cliente HTTP visual: configuramos el request (método, URL, headers, body) y Postman se encarga de enviarlo y mostrarnos la respuesta completa.

¿Para qué nos sirve?

- Probar endpoints antes de integrarlos al código Frontend.
- Ver exactamente qué devuelve una API (status code, headers, body)
- Debuggear por qué un request no funciona

Tip para el TP: Probar el endpoint en Postman, verificar que funciona y comprendes la estructura del request/response. Después de esto, escribimos el código Frontend (javascript en nuestro caso) para consumir la api. Si nuestro programa falla y en Postman funciona, el error esta en nuestro programa. Si también falla en Postman, el error es del backend.

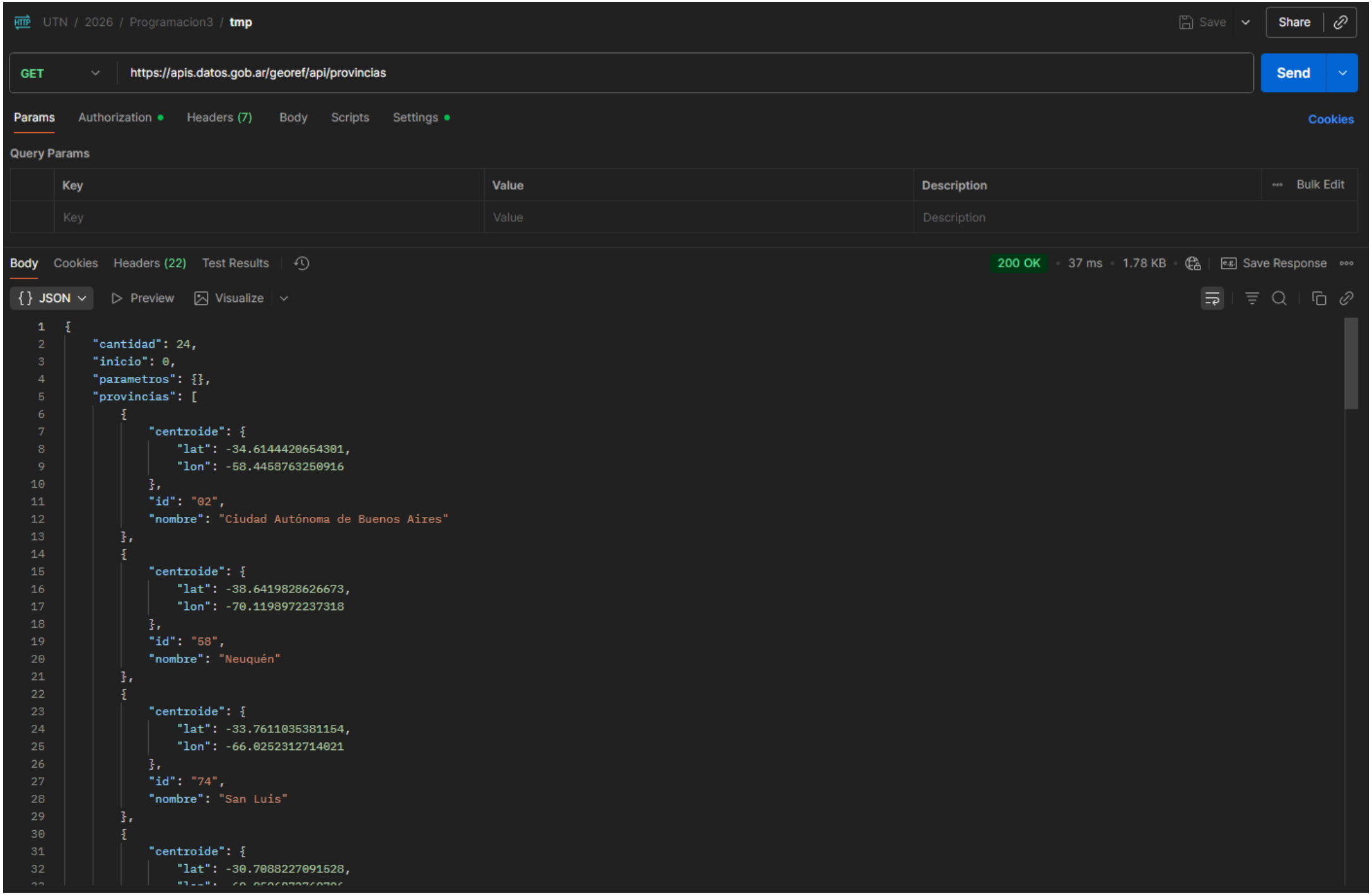


POSTMAN

Postman: Haciendo un GET

Paso a paso — GET a una API:

1. Abrí Postman → New Request
2. Seleccioná el método GET
3. Pegá la URL: https://apis.datos.gob.ar/georef/api/provincias
4. Hacé click en Send
5. Observá la respuesta


UTN / 2026 / Programacion3 / tmp
Save
Share
GET
https://apis.datos.gob.ar/georef/api/provincias
Send
Params
Authorization
Headers (7)
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
Cookies
Headers (22)
Test Results
200 OK
37 ms
1.78 KB
Save Response
JSON
Preview
Visualize
{
"cantidad": 24,
"inicio": 0,
"parametros": {},
"provincias": [
{
"centroide": {
"lat": -34.6144420654301,
"lon": -58.4458763250916
},
"id": "02",
"nombre": "Ciudad Autónoma de Buenos Aires"
},
{
"centroide": {
"lat": -38.6419828626673,
"lon": -70.1198972237318
},
"id": "58",
"nombre": "Neuquén"
},
{
"centroide": {
"lat": -33.7611035381154,
"lon": -66.0252312714021
},
"id": "74",
"nombre": "San Luis"
},
{
"centroide": {
"lat": -30.7088227091528,

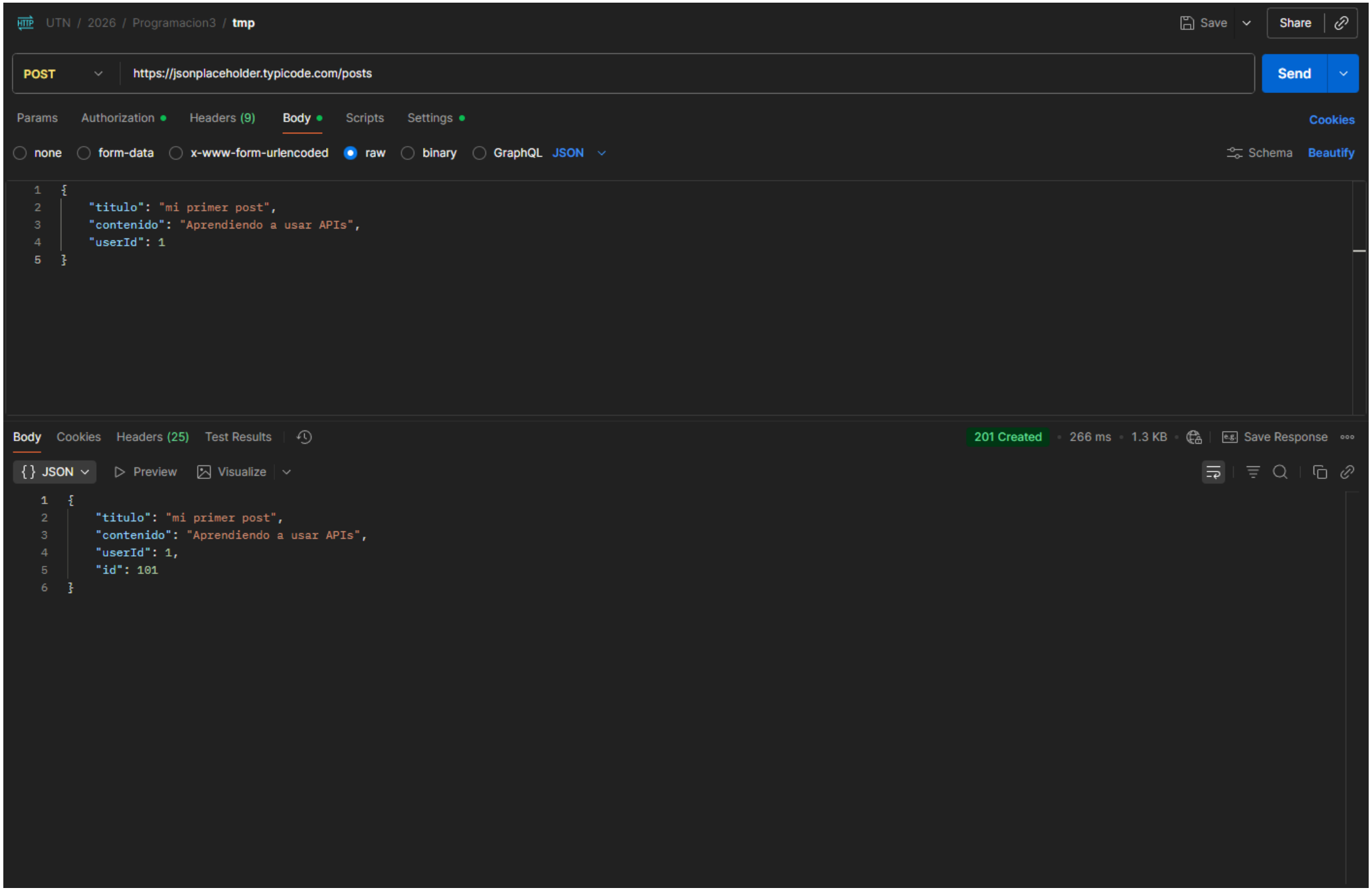
Postman: Haciendo un POST

Paso a paso — POST con body JSON:

Usamos https://jsonplaceholder.typicode.com/posts (API de prueba pública que simula respuestas reales)

1. Método: POST
2. URL: https://jsonplaceholder.typicode.com/posts
3. Ir al tab Body → seleccionar raw → formato JSON
4. Escribir el body:

```json
{
  "titulo": "Mi primer post",
  "contenido": "Aprendiendo a usar APIs",
  "userId": 1
}
```


UTN / 2026 / Programacion3 / tmp
Save
Share
POST
https://jsonplaceholder.typicode.com/posts
Send
Params
Authorization
Headers (9)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"titulo": "mi primer post",
"contenido": "Aprendiendo a usar APIs",
"userId": 1
}
Body
Cookies
Headers (25)
Test Results
201 Created
266 ms
1.3 KB
Save Response
JSON
Preview
Visualize
{
"titulo": "mi primer post",
"contenido": "Aprendiendo a usar APIs",
"userId": 1,
"id": 101
}

APIs Web para probar:

- https://dolarapi.com/docs/argentina/
- https://argentinadatos.com/docs/
- https://apis.datos.gob.ar/georef/api/provincias
- https://apis.datos.gob.ar/georef/api/municipios?provincia=Santa Fe&max=363



Programación 3 Frontend - 2026